

OTLOBLY

Food Ordering System

PROJECT DOCUMENTATION

Prepared by: Ahmed Ameer, Islam Ayman

Supervised by: Eng. Mariam Malak, Eng. Alhassan Kamel

Objective

The objective of this project is to develop a highly scalable, secure, and user-friendly web-based food ordering platform. **Otlobly** bridges the gap between consumers and restaurant administration, ensuring seamless order processing, secure payment handling, and efficient administrative oversight.

Technologies Used

Backend

Python

Flask

SQLAlchemy ORM

Database

MySQL

Frontend

HTML5

CSS3

Bootstrap

System Overview

Otlobly is built on a multi-tier architecture supporting discrete roles for standard users and system administrators. The platform provides an intuitive interface for clients to browse products, manage their carts, and execute transactions. Simultaneously, it offers a secure administrative dashboard for monitoring system health, managing personnel, and updating inventory dynamically. Data integrity is maintained strictly through relational mappings and cascading deletion rules.

System Features

User Module

- **Authentication:** Secure registration and login workflows.

- **Dashboard Access:** Personalized view of account status and history.
- **Order Management:** Complete CRUD (Create, Read, Update, Delete) capabilities for active orders.
- **Financial Transactions:** Secure interface for submitting order payments.

Administrator Module

- **User Management:** View registered accounts and execute account removals.
- **Cascading Deletions:** Deleting a user automatically purges their associated orders and payments to maintain strict data hygiene.
- **Order Oversight:** System-wide visibility into all placed orders.
- **Payment Management:** Ability to view and delete payment records.
- **Catalog Control:** Dynamic insertion of new food products into the database.

Database Design

The system relies on a strictly normalized relational database structure consisting of four primary entities.

Table Definitions

- **User:** Stores authentication credentials, contact information, and role-based access control flags (User vs. Admin).
- **Product:** Contains catalog information including product name, description, and pricing details.
- **Order:** Records transactional intent, linking users to specific products alongside timestamp and status attributes.
- **Payment:** Logs financial transactions, intrinsically tied to specific orders to verify completion status.

Entity Relationships

- **User → Order** **1:N** A single user can place multiple orders over time. Each order is strictly bound to one user.
- **Product → Order** **1:N** A specific product can appear across multiple different orders from various users.
- **Order → Payment** **1:0..1** An order can exist without a payment (pending), or possess exactly one successful payment record.

ERD Description (Logical Structure)

The Entity-Relationship architecture centers around the **Order** table, which acts as a transactional junction. The **Order** table contains foreign keys referencing both **User(id)** and **Product(id)**. The **Payment** table maintains a foreign key pointing back to **Order(id)** with a unique constraint to enforce the 1:0..1 relationship. Cascading rules are enforced at the database level on the **User(id)** foreign key within the **Order** table, ensuring that the removal of a primary **User** entity inherently resolves all dependent transactional data.

Data Flow Description

User Workflow

A visitor begins by interacting with the authentication layer to register or log in. Once authenticated, the system provisions a session and routes the user to their dashboard. The user fetches the product catalog, selects items, and dispatches a Create Order request. Following order generation, the user is prompted to initialize a payment. Upon successful payment verification, the system updates the order status.

Admin Workflow

Administrative personnel authenticate via the same login layer but are routed to the Admin Dashboard based on role evaluation. From this control center, the admin retrieves system-wide metrics. They can issue product insertion commands to expand the catalog, or issue deletion

commands targeting specific users, orders, or payments. Deletion commands trigger backend cascading protocols, ensuring no orphaned records remain.

API Endpoints

MODULE	METHOD	ENDPOINT	DESCRIPTION
Auth	POST	/auth/register	Creates a new user account.
Auth	POST	/auth/login	Authenticates and establishes session.
User	GET	/user/dashboard	Retrieves personalized metrics.
Orders	POST	/orders/create	Generates a new order.
Orders	PUT	/orders/update/ <id>	Modifies an existing pending order.
Orders	DELETE	/orders/delete/ <id>	Cancels and removes an order.
Payments	POST	/payments/process	Executes payment for an order.
Admin	GET	/admin/users	Lists all registered users.
Admin	DELETE	/admin/users/<id>	Removes user (cascades data).
Admin	POST	/admin/products/ add	Inserts a new product to catalog.

Conclusion

The Otlobly Food Ordering System provides a robust, scalable architecture utilizing modern web standards. By clearly separating administrative controls from user functionalities and enforcing strict relational database rules, the platform guarantees data integrity, secure transactional flows, and an intuitive user experience. The use of Flask paired with SQLAlchemy ensures future extensibility and low maintenance overhead.